Neural Networks 178 (2024) 106472

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

GeneWorker: An end-to-end robotic reinforcement learning approach with collaborative generator and worker networks

Hao Wang [a,b], Hengyu Man [a,c], Wenxue Cui [a,*], Riyu Lu [a,b], Chenxin Cai [a], Xiaopeng Fan [a,c]

[a] Faculty of Computing, Harbin Institute of Technology, Harbin, 150001, Heilongjiang, China
[b] College of Engineering, City University of Hong Kong, Kowloon, 999077, Hong Kong Special Administrative Region of China
[c] Pengcheng Laboratory, Shenzhen, 518055, Guangdong, China

ARTICLE INFO

Keywords:
Robotic reinforcement learning
Neural skills
Network parameters
Collaborative inferences

ABSTRACT

Reinforcement learning aided by the skill conception exhibits potent capabilities in guiding autonomous agents toward acquiring meaningful behaviors. However, in the current landscape of reinforcement learning, a skill is often merely a rudimentary abstraction of a sequence of primitive actions, serving as a component of the input to policy networks with fixed network parameters. This rigid methodology presents obstacles when attempting to integrate with burgeoning techniques such as meta-learning and large language models. To address this issue, we introduce a unique neural skill representation that abstracts the activation of neurons in each neural layer. Based on this, a novel end-to-end robotic reinforcement learning algorithm is proposed, in which two sub-networks, i.e., generator and worker networks, implement collaborative inferences via neural skills. Specifically, the generator produces a series of multi-spatial neural skills, providing efficient guidance for subsequent decision-making; by integrating these skills, the worker can determine its own network weights and biases to cope with various environmental conditions. Therefore, actions can be sampled with flexibly changeable network parameters through the collaboration between generator and worker networks. The experiments demonstrate that GeneWorker can achieve a mean success rate of over 90.67% on continuous robotic tasks and outperforms previous state-of-the-art methods by a minimum of 54% on the pick-and-place task.

1. Introduction

Artificial intelligence (AI) has recently empowered various industries and propelled humankind into the intelligent era. Notably, the employment of reinforcement learning (RL) enables autonomous agents, like robots and self-driving vehicles, to learn diverse behaviors to deal with arbitrary tasks (Agrawal, Nair, Abbeel, Malik, & Levine, 2016; Levine, Pastor, Krizhevsky, Ibarz, & Quillen, 2018; Lillicrap et al., 2015; Mnih et al., 2015; Pathak et al., 2018; Silver et al., 2017). Unlike supervised learning, which requires a given dataset before training the model, RL methods usually explore environments and leverage the collected experiences to learn, which is more similar to the learning styles of infants and other animals. It is even claimed that RL with an appropriate reward function is sufficient for AI to achieve general intelligence like humans and other conscious organisms (Silver, Singh, Precup, & Sutton, 2021).

RL algorithms incorporating skills empower intelligent agents to work flexibly on a multitude of complex tasks. Nevertheless, in the majority of contemporary methodologies, a skill is merely depicted as a feature vector that abstracts a sequence of primitive actions, serving as a part of the input to a neural network with fixed parameters (Campos et al., 2020; Gregor, Rezende, & Wierstra, 2016; Tessler, Givony, Zahavy, Mankowitz, & Mannor, 2017; Zhang, Yu, & Xu, 2021). This inflexible approach makes it difficult to integrate with emerging techniques such as meta-learning, large language models, and Transformers. Inspired by the neural mechanisms of the brain, we aspire to devise an approach that focuses on training the network to generate policies, diverging from conventional RL approaches of directly training policies. Each brain region has its own unique function. Different brain regions will be frequently activated during the process of solving a task. For example, when learning via recorded lectures, visual information is interpreted primarily in the brain's occipital lobe (Gupta, 2017), while the temporal lobe is responsible for understanding language (Barense, Warren, Bussey, & Saksida, 2016); the primary motor cortex can control muscles to perform movements, such as taking notes and adjusting sitting postures (Brodal, 2016).

* Corresponding author.
E-mail addresses: ho.wong@cityu.edu.hk (H. Wang), manhengyu@hotmail.com (H. Man), wxcui@hit.edu.cn (W. Cui), riyu_Lu@hotmail.com (R. Lu), chx.cai@outlook.com (C. Cai), fxp@hit.edu.cn (X. Fan).

https://doi.org/10.1016/j.neunet.2024.106472
Received 5 November 2023; Received in revised form 7 May 2024; Accepted 16 June 2024
Available online 18 June 2024
0893-6080/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

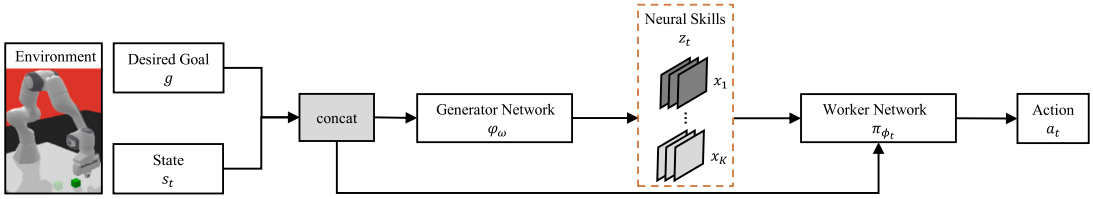


Fig. 1. The whole framework of GeneWorker. The desired goal g and the state s_t are concatenated as the input to the generator network φ and the worker network π independently. The generator network φ, with its network parameters ω, can generate a group of neural skills z_t where each neural skill x_k contained in z_t corresponds to a neural layer in the worker network π. Then, the neural skills z_t are integrated by the worker and converted to its own self-adaptive network parameters ϕ_t.

Hence, how can we mimic this neural mechanism to facilitate robotic learning, thereby enabling a corresponding neural network to process the information?

A straightforward way is to train several network models, and each of them represents a specific brain region. Nonetheless, training a set of networks is quite computationally inefficient (Su et al., 2020), and it is also difficult to unambiguously determine which network is suitable for processing the current input information. An alternative way is to design a method allowing a network model with self-adaptive parameters to process information. The network parameters, which change dynamically according to the input information, can be used to represent different brain regions. In this paper, the stimulation of neurons in a neural layer is treated as a multi-spatial neural skill, and an end-to-end robotic RL algorithm is further proposed, in which generator and worker networks implement collaborative inferences using these skills. As illustrated in Fig. 1, the generator produces a series of multi-spatial neural skills to guide the following decision-making efficiently. At the same time, the worker can transform these neural skills to its own self-adaptive network weights and biases to tackle varying desired goals and states. The experiments showcase that our method is able to solve tasks with a mean success rate of over 90.67% and outperforms prior state-of-the-art methods by at least 54% on the PandaPickAndPlace task.

Our contributions are three-fold:

- We propose an end-to-end robotic reinforcement learning model including **gene**rator and **worker** networks (GeneWorker), outperforming several prior algorithms. Through the collaboration between generator and worker networks, our model is able to cope with various environmental conditions with self-adaptive network parameters even after training.
- For the generator network, a novel multi-spatial neural skill representation is proposed to abstract the neuronal activity at each neural layer, and it is produced for each neural layer of the worker network, which can better reflect the role of actual manipulative skills at the neural level.
- For the worker network, it is able to understand the multi-spatial neural skills produced by the generator and further translate these skills to its own self-adaptive network parameters. In this way, it is beneficial for the worker to handle varying desired goals and states with dynamic network weights and biases even after completing the training.

This paper is structured as follows. Firstly, we briefly introduce the preliminaries and the prior research works in Section 2 and Section 3, respectively. After that, the proposed GeneWorker method is described in Section 4. In Section 5, the experimental results and analysis are reported. Finally, we conclude this study in Section 6.

2. Preliminaries

RL is a type of machine learning where an agent incrementally learns to make optimal decisions by interacting with an environment to achieve a specific goal. The learning process is driven by the consequences of the agent's actions, thereby adjusting its behavior in response to the rewards or penalties it receives.

A Markov Decision Process (MDP) is a mathematical model used to describe an environment for reinforcement learning. The goal-conditioned MDP framework includes states $\mathcal{S}$, actions $\mathcal{A}$, goals $\mathcal{G}$, transition probabilities $p(s_{t+1}|s_t, a_t)$, a discount factor $\gamma \in [0, 1]$, and a reward function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{G} \to \mathbb{R}$, which align with the components of an RL problem.

In the context of RL, the agent interacts with the environment in a series of timesteps. At each timestep, the agent is in a state $s_t \in \mathcal{S}$, chooses an action $a_t \in \mathcal{A}$, receives a reward r_t, and transitions to a subsequent state $s_{t+1} \in \mathcal{S}$. The goal-conditioned MDP is represented as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{G}, p, \gamma, r)$.

3. Related works

Nowadays, the application of RL to robots solving complex manipulation tasks is still a challenging topic. We roughly divide the previous research work into two categories: flat RL and hierarchical RL algorithms.

3.1. Flat reinforcement learning algorithms

The classical RL algorithms applied to robotic learning are mainly based on actor–critic architecture (Barto, Sutton, & Anderson, 1983; Sutton, McAllester, Singh, & Mansour, 1999). A2C and A3C (Mnih et al., 2016) adopt the advantage function and focus on parallel training to reduce the variance of estimation, improving the learning efficiency and stabilizing the training. DDPG (Lillicrap et al., 2015) combines the advantages of DQN (Mnih et al., 2015) and DPG (Silver et al., 2014), which adopts the use of a replay buffer and target networks to address the difficulty of convergence. TD3 (Fujimoto, Hoof, & Meger, 2018) algorithm is an advanced algorithm based on DDPG, which utilizes twin action-value networks and delayed updating for the policy and the target networks. SAC (Haarnoja, Zhou, Abbeel, & Levine, 2018; Haarnoja et al., 2019) uses an additional soft entropy term and the adaptive temperature hyperparameter adjustment, considerably boosting exploration.

Inverse RL methods (Finn, Levine, & Abbeel, 2016; Fu, Luo, & Levine, 2018; Ho & Ermon, 2016; Wulfmeier, Ondruska, & Posner, 2015), usually regarded as a type of imitation learning, try to approximate reward functions from expert demonstrations and then conduct classical RL methods based on these reward functions. Wang et al. (2016) present a dueling network architecture to separately estimate state values and state-dependent action advantages. Hindsight Experience Replay (HER) (Andrychowicz et al., 2017) improves the learning efficiency of RL in the setting of sparse rewards. Singh, Yang, Hartikainen, Finn, and Levine (2019) propose the VICE-RAQ method adopting active queries into RL. Pathak, Agrawal, Efros, and Darrell (2017) propose an Intrinsic Curiosity Module (ICM) that generates an intrinsic reward signal guiding classical RL algorithms to explore environments. RIG method (Nair et al., 2018) applies VAE (Kingma & Welling, 2022) to RL to help generate imagined goals; then, Pong et al. (2020) propose the Skew-Fit algorithm to maximize the entropy of the goal distribution for improving the performance of RIG. MUSIC (Zhao, Gao, Abbeel, Tresp, & Xu, 2021) uses mutual information between the

agent state and the surrounding state for RL to encourage the agent to have maximum control over its surroundings.

These methods usually leverage flat neural network structures. Many of them propose novel modules or approaches to improve performance. Nevertheless, unlike our method, they do not reflect the neural stimulation activated by manipulative skills, and many of them do not even introduce the concept of skill.

3.2. Hierarchical reinforcement learning algorithms

In recent years, hierarchical RL has become a popular research area in RL. HAC (Levy, Konidaris, Platt, & Saenko, 2019) is able to learn multiple levels of policies simultaneously by consisting of a particular hierarchical architecture and two types of hindsight transitions, which overcomes the non-stationary issues that hinder the joint learning of multiple levels in parallel. Similarly, HIRO (Nachum, Gu, Lee, & Levine, 2018) introduces an approach called off-policy correction to address the non-stationary problems. RIS (Chane-Sane, Schmid, & Laptev, 2021) leverages a separate high-level policy to predict imagined subgoals that are incorporated to facilitate the training. Qian, You, Zhou, Xu, and He (2023) propose a goal-conditioned RL algorithm combined with Disentanglement-based Reachability Planning (REPlan), which introduces a Disentangled Representation Module (DRM) and a Reachability Discrimination Module (REM) to equip agents with the capability of reaching distant goals. H-DRLN (Tessler et al., 2017) incorporates Deep Skill Networks (DSN) to learn reusable skills. DSG (Bagaria, Senthil, & Konidaris, 2021) builds a graph representation of continuous Markov decision processes, where nodes are skill subgoals and edges are skill policies. Jain, Iscen, and Caluwaerts (2019) propose a hierarchical framework in which the high-level policy issues latent commands and durations for guiding the low-level policy to control robots. DIAYN method (Eysenbach, Gupta, Ibarz, & Levine, 2019) learns valuable skills by maximizing the mutual information between states and skills. DADS (Sharma, Gu, Levine, Kumar, & Hausman, 2020) learns behaviors that are predictable by dynamics models. HiPPO (Li, Florensa, Clavera, & Abbeel, 2020) utilizes a new hierarchical policy gradient and a method for training time abstractions, ensuring the quality of skills learned. HIDIO (Zhang et al., 2021) is able to learn task-agnostic options while learning to schedule them to solve the environment tasks.

Hierarchical RL has the potential to boost learning in various sparse reward tasks since it introduces a flexible network framework. However, multi-level hierarchies are usually problematic to converge due to non-stationary state transitions. Similar to flat RL methods, these hierarchical methods also do not extend the manipulative skills to the neural level like our neural skills, and are still difficult to integrate with emerging technologies such as meta learning and large language models.

4. The proposed GeneWorker

Our study aims to propose an approach for agents to solve sparse reward goal-conditioned tasks without supervision. The proposed method includes two main components: a generator network and a worker network. To the best of our knowledge, the skill representation is used at the neural level for the first time while each skill abstracts the activity of neurons in each neural layer. The proposed generator network generates a series of neural skills for the worker network. After receiving them, the worker network first translates these skills to its own dynamic weights and biases and then outputs the action a_t. In this section, the overview of the self-adaptive RL model is introduced in Section 4.1 at first. The proposed generator and worker networks are described in Section 4.2 and Section 4.3, respectively. Finally, the whole proposed GeneWorker algorithm is presented in Section 4.4.

4.1. Self-adaptive reinforcement learning model

The standard goal-conditioned RL focuses on learning a policy $\pi_\phi(a_t|s_t, g)$ maximizing the expected sum of rewards where ϕ represents the network parameters of model π. The procedure of taking action a_t can be described as the following:

$$a_t \sim \pi_\phi(a_t|s_t, g). \tag{1}$$

In this way, once the network training is completed, fixed network parameters are used to cope with the variety of goals and states, which means that it is like always using the same area of the brain to handle the different environmental conditions. However, in reality, it is impractical for humans to deal with the diversity of complicated tasks with just one brain region. Even when a person solves one task, several distinct brain regions will be activated. Thus, to mimic this pattern of humans, self-adaptive methods need to modify the procedure of sampling an action from (1) to the following:

$$a_t \sim \pi_{\phi_t}(a_t|s_t, g), \tag{2}$$

where parameters ϕ_t are self-adaptively dependent on the desired goal g and the current state s_t instead of being unchanged for all inputs after completing training.

4.2. Generator network for producing neural skills

Instead of previous algorithms concentrating on manipulative skills, our method focuses on skills at the neural level, which is why we call them neural skills. In our approach, the network φ generating neural skills is called a *generator*. Hence, the group of neural skills z_t can be described as the following:

$$z_t = \varphi_\omega(s_t, g), \tag{3}$$

where ω stands for the generator network parameters. As shown in Fig. 2, the function of the generator network is mapping desired goals g and current states s_t to multi-spatial neural skills z_t for the worker network. In order to make it convenient for the worker to convert neural skills into its own self-adaptive network parameters, one of the most straightforward approaches is to design a neural skill containing network parameters for a neural layer. As fully-connected layers are the essential components of the worker network, the generator should generate two types of parameters, i.e., weights and biases, for the worker. Inspired by Klocek et al. (2019) and Su et al. (2020), the generator network in our model is comprised of N fully-connected layers and K generating branches where N and K are the hyperparameters. Each branch produces a neural skill x_k, including weights w_k and biases b_k, for an individual fully-connected layer in the worker network. The generated weights are from convolution followed by a reshape operation, while the generated biases are simply from a fully-connected layer. All the generated neural skills $x_{k,k=1,2,\ldots,K}$ make up the group of neural skills z_t and are conveyed to the worker.

4.3. Worker network for integrating neural skills

The policy network π in our model is called a *worker*. At first, this network needs to transform the multi-spatial neural skills z_t, which are received from the generator, to a series of self-adaptive network parameters ϕ_t by a transformation function $f(\cdot)$. Thus, the parameters can be represented as:

$$\phi_t = f(z_t). \tag{4}$$

As mentioned above, our design of the generator network facilitates the worker to transform neural skills to its own network parameters. Therefore, we directly use a simple transformation function so that:

$$f(z_t) = z_t. \tag{5}$$

However, a more efficient network design and more powerful transformation function could be left for deep study in future work. As presented in Fig. 3, the worker network is made up of K fully-connected layers ($K-2$ hidden layers with ReLU and 2 output layers generating

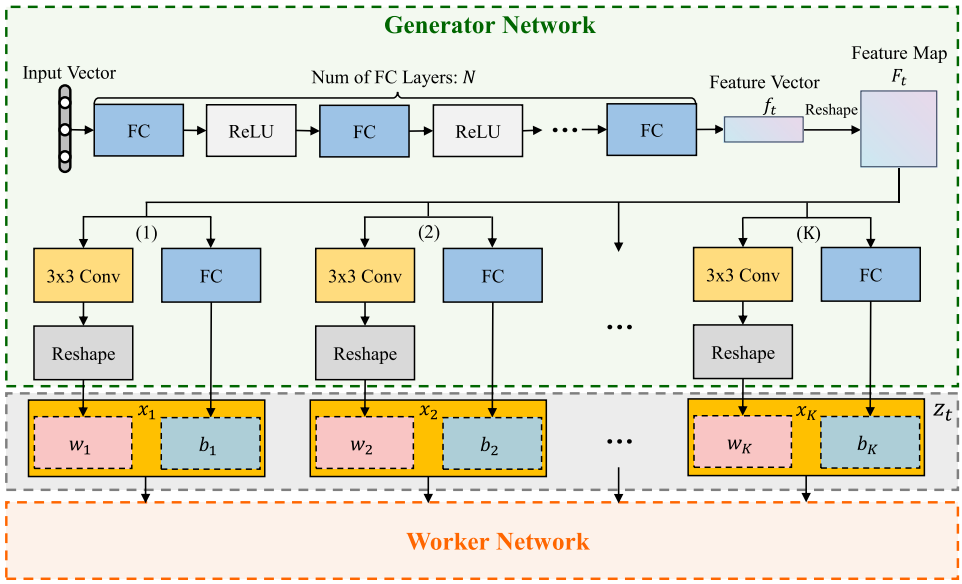


Fig. 2. The pipeline of the generator network. The generator network produces neural skills z_t for the worker network. At first, the input vector, the concatenation of the desired goal g and the state s_t, is mapped to a feature vector f_t by N fully-connected layers, i.e., $N-1$ hidden layers with ReLU and an output layer outputting f_t. Next, the feature vector f_t is reshaped to a single-channel feature map F_t. Then, the generating branches generate neural skills z_t for fully-connected layers of the worker network.

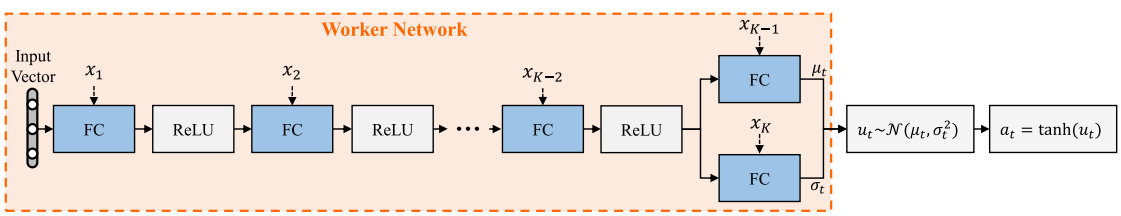


Fig. 3. The pipeline of the worker network. There are K fully-connected layers in the worker network, which are $K-2$ hidden layers with ReLU and 2 output layers producing a Gaussian. After they are equipped with weights $w_{k,k=1,2,\ldots,K}$ and biases $b_{k,k=1,2,\ldots,K}$, which are converted from the neural skills $z_t = \{x_1, x_2, \ldots, x_K\}$, they will be able to map the desired goal g and the current state s_t to a Gaussian as the action distribution.

a Gaussian), which is equal to the number of generating branches in the generator. This network is used to map the concatenation of goal g and state s_t to an unbounded Gaussian $\mathcal{N}(\mu_t, \sigma_t^2)$ as the action distribution. Nevertheless, in practice, the actions are usually within a bounded interval. Therefore, an invertible squashing function, i.e., tanh, is applied to the Gaussian samples. Combining (2), (3), (4), and (5), the whole GeneWorker model can be formulated as:

$$a_t \sim \pi_{\varphi_\omega(s_t,g)}(a_t|s_t,g). \tag{6}$$

Through the collaboration between the generator and worker networks, our experiments demonstrate that GeneWorker performs competitively on continuous goal-conditioned robotic tasks and can also be successfully extended to discrete control tasks with excellent performance.

4.4. GeneWorker algorithm

In our model, neural networks are leveraged to approximate two soft Q-functions, $Q : \mathcal{S} \times \mathcal{A} \times \mathcal{G} \to \mathbb{R}$, with parameters θ_1 and θ_2, respectively, and two target soft Q-functions with parameters $\bar{\theta}_1$ and $\bar{\theta}_2$, respectively. Similar to SAC (Haarnoja et al., 2019), each of $\theta_{i,i\in\{1,2\}}$ is trained independently to minimize the loss $L_Q(\theta_i)$,

$$L_Q(\theta) = \mathbb{E}_{(s_t,a_t,g)\sim\mathcal{D}}[(Q_\theta(s_t,a_t,g) - \bar{Q}_{\bar{\theta}}(s_t,a_t,g))^2], \tag{7}$$

in which

$$\bar{Q}_{\bar{\theta}}(s_t,a_t,g) = r(s_t,a_t,g) + \gamma\mathbb{E}_{s_{t+1}\sim p}[V_{\bar{\theta}}(s_{t+1})] \tag{8}$$

and

$$V_{\bar{\theta}}(s_t) = \mathbb{E}_{a_t\sim\pi_{\phi_t}}[\min_{i=1,2} Q_{\bar{\theta}_i}(s_t,a_t,g) - \alpha\log\pi_{\phi_t}(a_t|s_t,g)] \tag{9}$$

where $\mathcal{D}$ denotes a replay buffer, $r(\cdot)$ represents a reward function, γ stands for a discount factor, and α is the temperature parameter. After soft Q-function networks are updated, the generator network parameters ω are optimized by maximizing the objective function,

$$J_\varphi(\omega) = \mathbb{E}_{(s_t,g)\sim\mathcal{D}}[\mathbb{E}_{a_t\sim\pi_{\phi_t}}[\min_{i=1,2} Q_{\theta_i}(s_t,a_t,g) - \alpha\log\pi_{\phi_t}(a_t|s_t,g)]], \tag{10}$$

where $\phi_t = f(\varphi_\omega(s_t,g))$ according to (3) and (4). As in SAC, we update the temperature parameter α by maximizing $J(\alpha)$,

$$J(\alpha) = \mathbb{E}_{(s_t,g)\sim\mathcal{D}}[\mathbb{E}_{a_t\sim\pi_{\phi_t}}[\alpha\bar{\mathcal{H}} + \alpha\log\pi_{\phi_t}(a_t|s_t,g)]], \tag{11}$$

where $\bar{\mathcal{H}}$ denotes the entropy target. Finally, target soft Q-function network parameters $\bar{\theta}_{i,i\in\{1,2\}}$ are updated by

$$\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i, i \in \{1,2\}, \tag{12}$$

where τ is a target smoothing coefficient. The whole procedure of the GeneWorker algorithm is listed in Algorithm 1.

5. Experiments

The environments used for interacting with autonomous agents are introduced in Section 5.1. The experimental results are reported and analyzed in Section 5.2. In Section 5.3, the neural skills and network structures of GeneWorker are analyzed. In Section 5.4, GeneWorker is extended to discrete control tasks, and its excellent performance is reported and discussed.

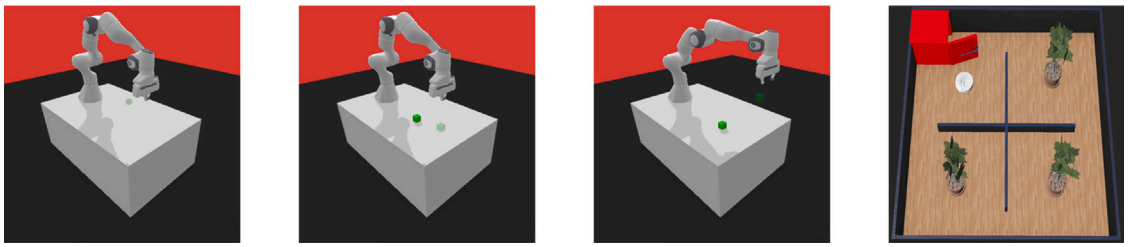


Fig. 4. The four goal-conditioned robotic tasks we evaluate on. There are three robotic manipulation tasks in Panda Gym and a navigation-manipulation task constructed on the Webots simulator. From left to right: *PandaReach*, *PandaPush*, *PandaPickAndPlace*, and *WebotsDoor*.

Algorithm 1: GeneWorker.

Input: θ_1, θ_2, ω
Output: θ_1, θ_2, ω
$\bar{\theta}_i \leftarrow \theta_i$, $i \in \{1,2\}$
$\mathcal{D} \leftarrow \emptyset$
for each episode **do**
  Sample a desired goal g from the environment.
  for each interaction **do**
    $z_t = \varphi_\omega(s_t, g)$
    $\phi_t = f(z_t)$
    $a_t \sim \pi_{\phi_t}(a_t|s_t, g)$
    $s_{t+1} \sim p(s_{t+1}|s_t, a_t)$
    $r_t \leftarrow r(s_t, a_t, g)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r_t, s_{t+1}, g)$
    if train **then**
      Minimize $L_Q(\theta_i)$, $i \in \{1,2\}$.
      Maximize $J_\varphi(\omega)$.
      Maximize $J(\alpha)$.
      Update $\bar{\theta}_i$, $i \in \{1,2\}$.
    end if
  end for
end for

5.1. Environments

To evaluate the proposed method against several prior algorithms, we use four goal-conditioned robotic tasks with a sparse reward function, as presented in Fig. 4 (Gallouédec, Cazin, Dellandréa, & Chen, 2021; Michel, 2004). The first three environments simulate the Panda robot arm of Franka Emika for reaching, pushing, and picking & placing tasks, respectively, while the last environment based on the Webots simulator emulates iRobot Create robot in a navigation-manipulation task. An episode of each task is made of 50 interactions. At the beginning of each episode, a desired goal for this episode is randomly generated. All the tasks are implemented with a sparse reward function: a reward of 0 is obtained if the object is at the desired position within a given tolerance and −1 otherwise. In the navigation-manipulation task, i.e., WebotsDoor, the desired goal is the target position of the door. The observation contains the positions of the door and the robot, while the action is composed of the robot's movement in position coordinates. The door is initially open at 90 degrees, and the robot is initialized to any potential location in the room. The robot must first navigate to the door and then maneuver the door to the target position. It is worth noting that the door may never return to the target position if it is closed too much by the robot, which means the task is already doomed to fail even though the episode is not finished.

5.2. Experimental results

We compare our method with several prior methods, including SAC (Haarnoja et al., 2019), HIDIO (Zhang et al., 2021), ICM (Pathak et al., 2017), and MUSIC (Zhao et al., 2021). Here, the ICM method is SAC (Haarnoja et al., 2019) augmented by the intrinsic curiosity module, and the MUSIC is the MUSIC-r version. All methods run with HER (Andrychowicz et al., 2017). Without loss of generality, we evaluate algorithms on both environments with and without surrounding objects. Therefore, MUSIC is not conducted on the PandaReach task since there is no mutual information between agent states and surrounding states for MUSIC to maximize. The experimental details are presented in Appendix A.

Fig. 5 shows the mean success rates of all methods on four goal-conditioned tasks. On PandaReach, all methods display competitive performance, although HIDIO takes slightly longer to start performing well than other methods. On PandaPush and WebotsDoor, SAC, MUSIC, ICM, and GeneWorker have similar performance, while ICM slightly fluctuates at the beginning of training on WebotsDoor. On PandaPickAndPlace, the performance of MUSIC is similar to that of ICM, and SAC is relatively non-competitive. GeneWorker has a rapid increase in performance after about 1,050,000 timesteps and achieves a success rate of over 90% around the 1,450,000th timestep. At that time, ICM and MUSIC methods have just reached a success rate of approximately 30%. In contrast, HIDIO is less competitive than other methods on PandaPush, PandaPickAndPlace, and WebotsDoor tasks. Table 1 shows the overall performance evaluation on four tasks. As we can see, all methods can perform perfectly on PandaReach with a mean success rate of 100% since this task could be the easiest one among these four tasks. On PandaPush, our method achieves a higher mean success rate than the second-best method by 2%. On WebotsDoor, this value is 1.33%. On PandaPickAndPlace, the most complex of these four tasks, GeneWorker achieves a mean success rate of 90.67%, which is higher than SAC, HIDIO, ICM, and MUSIC by 76.67%, 82.67%, 54%, and 57.34%, respectively. This result demonstrates that GeneWorker is highly competitive in solving sophisticated tasks. As one of the hierarchical RL methods, the performance of HIDIO will drop noticeably as the hardness of the task increases. In comparison, the performance of other algorithms does not experience such significant degradation. On PandaPickAndPlace, the mean success rate of HIDIO is even as low as 8.00%, again indicating the training difficulty of hierarchical RL on sophisticated tasks. Notably, the performance of our algorithm fluctuates minimally across different tasks, which demonstrates the robustness of our method to tasks with varying difficulty.

In summary, our method is able to reach a higher mean success rate than other methods in all four tasks. In addition, the standard deviation of our method is also tiny, which indicates that our method is robust to the diversity of environmental conditions in each task.

5.3. Analysis of neural skills and network structures

For conveniently observing the variations of neural skills when performing different actions, the mean and standard deviation of each weight and bias in neural skills on PandaReach are visualized. The statistics are based on 150 random samples of state-goal pairs where each 25 of 150 samples can be mapped to actions in a positive or negative direction along one of the x, y, and z axes. As depicted in

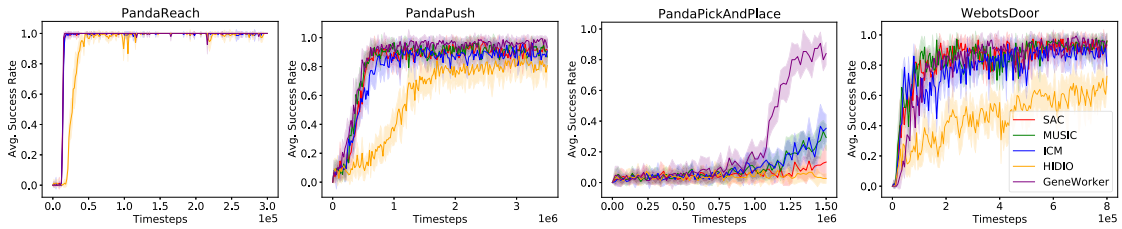


Fig. 5. Comparisons of all methods on four goal-conditioned tasks. The solid lines indicate the mean success rates, and the shaded areas describe the standard deviations.

Table 1
Overall performance evaluation on four tasks.

Method	PandaReach (%)	PandaPush (%)	PandaPickAndPlace (%)	WebotsDoor (%)
SAC (Haarnoja et al., 2019)	**100% ± 0%**	97.33% ± 3.27%	14.00% ± 10.52%	98.00% ± 3.06%
HIDIO (Zhang et al., 2021)	**100% ± 0%**	89.33% ± 6.80%	8.00% ± 7.77%	72.67% ± 7.57%
ICM (Pathak et al., 2017)	**100% ± 0%**	96.00% ± 4.42%	36.67% ± 14.38%	94.00% ± 6.96%
MUSIC (Zhao et al., 2021)	–	96.67% ± 5.37%	33.33% ± 7.30%	98.00% ± 3.06%
GeneWorker (ours)	**100% ± 0%**	**99.33% ± 2.00%**	**90.67% ± 7.42%**	**99.33% ± 2.00%**

Appendix C, their mean values are distributed between approximately −2.05 and 1.3, implying that almost all weights and biases are activated when making decisions. Many weights and biases have relatively large standard deviations regardless of whether the generated actions are similar, which demonstrates that our model is able to cope with various environmental conditions by the neural skills with a wide range of variations.

In order to determine an appropriate combination of the generator and worker network structures, we ablate the number of fully-connected layers N and the number of generating branches K for the generator network as well as the number of neurons U in each hidden layer of the worker network on the less time-consuming task, i.e., PandaReach. The remaining experimental configurations are the same as those in Appendix A. Fig. 6 shows the mean success rates of each generator–worker structure at the 13,500th timestep, the first time a combination reaches the 100% success rate. It is easy to find that the performance of most models gradually decreases as the number of generating branches increases. The worker structures with a larger number of neurons in each hidden layer are also usually less competitive. These degradations occur because a complex structure of generating branches easily results in the accumulation of large error gradients in the backbone layers of the generator, thus leading to the exploding gradient problem. Therefore, it is indicated that a small-scale worker network structure is usually competitive in task solving. In summary, the findings mentioned above provide guidance for GeneWorker on selecting network structures.

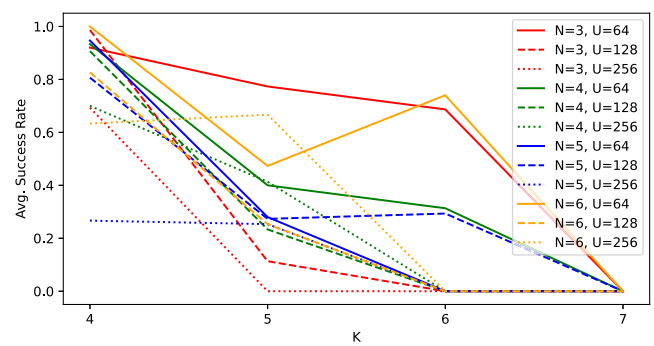
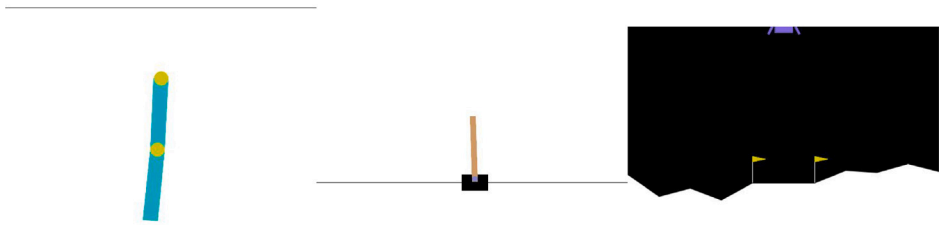


Fig. 6. The average success rates of each generator–worker structure at the 13,500th timestep.

5.4. Expansion to discrete reinforcement learning tasks

While GeneWorker is predominantly employed for continuous RL tasks in robotics, we have nonetheless extended its application to discrete control tasks. As shown in Fig. 7, the discrete tasks utilized for evaluation are three classic control tasks sourced from Gymnasium (Towers et al., 2023), which are Acrobot, CartPole, and LunarLander. The observation of the Acrobot environment is a 6-dimensional vector that provides information about the two rotational joint angles and their angular velocities. It has a discrete action space of 3, which represents the torque applied on the actuated joint between the two links. Every step that fails to achieve the target results in a penalty of −1. When the target height is reached, the process ends with a neutral reward of 0. The CartPole environment has a continuous observation space of 4 dimensions, indicating the cart position and velocity as well as the pole angle and angular velocity. It has a discrete action space of 2, signifying the direction of the fixed force applied to the cart. For each step in which the pole is maintained upright, a reward of +1 is allotted. The observation of the LunarLander environment is represented by an 8-dimensional vector, which includes information about the coordinates of the lander, its linear velocities, its angle, its angular velocity, and whether each leg is touching the ground. Its action space consists of four possible actions: doing nothing, firing the left orientation engine, firing the main engine, and firing the right orientation engine. Upon the completion of each step, a corresponding reward is bestowed. Upon the termination of each episode, a supplementary reward is also allocated, with −100 points for a crash and +100 points for a safe landing, respectively. The algorithms compared with GeneWorker are two classic algorithms applied to discrete control tasks, which are DQN (Mnih et al., 2015) and SAC (Christodoulou, 2019). The experimental details are presented in Appendix A.

As can be seen from Fig. 8, in the Acrobot task, the DQN method gradually reaches a high-performance level, whereas SAC and GeneWorker achieve a similar level of performance after undergoing a period of substantial fluctuations. In the CartPole task, after reaching their individual performance peaks, all three methods experience varying degrees of performance degradation. Then, they all eventually reach a good level of performance again. Notably, GeneWorker outperforms the other methods, achieving the highest performance level with an episode reward of 500. In the LunarLander task, the DQN method exhibits excellent performance rapidly, while SAC and GeneWorker methods subsequently reach a comparable level of performance. Ultimately, GeneWorker reaches the best performance level, with an approximate episode reward of 270; SAC follows closely, with an approximate episode reward of 265. These experimental results demonstrate that GeneWorker can be successfully extended to discrete control tasks with excellent performance.



(a) Acrobot (b) CartPole (c) LunarLander

Fig. 7. Three discrete control tasks leveraged for evaluation. (a) The Acrobot environment models a two-link robotic arm with an actuator at the elbow, where the goal is to swing the end of the lower link up to a target height. (b) The CartPole environment involves a pole attached to a cart moving along a frictionless track, with the aim to prevent the pole from falling. (c) The LunarLander environment simulates a landing spacecraft, where the goal is to land the spacecraft between two flags.

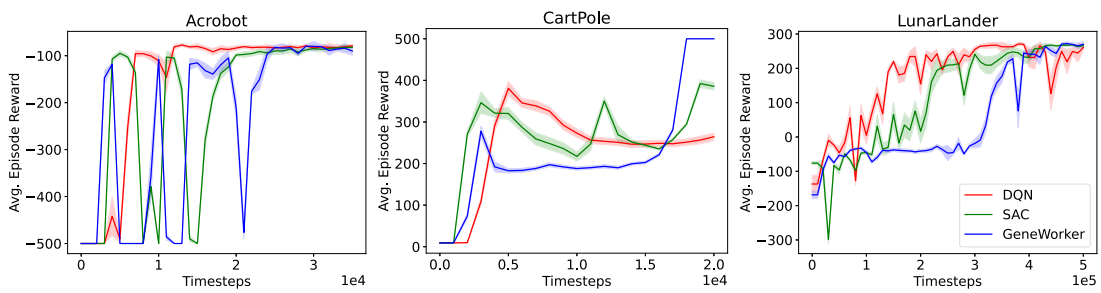


Fig. 8. Episode rewards of DQN, SAC, and GeneWorker on three discrete control tasks. The solid lines indicate the mean episode rewards, and the shaded areas describe the standard deviations.

6. Conclusions and discussions

RL, a type of learning-based control method, provides significant advantages in autonomous control. It (1) allows autonomous agents to quickly adapt to new environments and tasks; (2) can also generalize from past experiences to new situations; (3) can continually optimize the control policy based on feedback from the environment; (4) increases the autonomy of agents, reducing the need for human intervention; and (5) is particularly effective in handling complex tasks where mathematical modeling is challenging. In contrast, non-learning control approaches usually require manual reconfiguration, lack generalization capability, and do not offer ongoing optimization, necessitating significant human supervision. Therefore, the advantages of the RL method make it the preferred choice over non-learning methods in numerous applications.

In this paper, we propose an end-to-end robotic RL method with collaborative generator and worker networks. Instead of the common skill representations used in previous work, we consider a group of skills at the neural level for the policy network as neural skills. The generator network is able to produce neural skills in multiple spaces for the worker according to the environmental conditions. Next, our policy network, i.e., the worker, is able to transform these multi-spatial neural skills to its own self-adaptive network parameters and then handles the diversity of environmental states and desired goals. The experimental results suggest that our model equips agents with the competitive ability to face goal-conditioned environments.

In the future, we plan to further integrate GeneWorker with emerging technologies such as meta-learning and large language models to enhance its task performance and environmental universality. Ideas from control and design domains (Precup, Roman, & Safaei, 2021; Rigatos, Siano, Selisteanu, & Precup, 2017; Ucgun, Okten, Yuzgec, & Kesler, 2022; Wu, Xie, & Hou, 2022) can be coalesced with our approach for practical application in robotic engineering. In addition, our method also has limitations that need to be overcome. As listed in Appendix B, GeneWorker entails a slightly larger number of network parameters compared to benchmark methods. Furthermore, since the neural skills output by the generator network need to be transformed into the network parameters of the worker network in the process of forward propagation, the training time of the GeneWorker method is longer than benchmark methods. Therefore, future work should also aim to address these limitations.

CRediT authorship contribution statement

Hao Wang: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Validation, Visualization, Writing – original draft, Writing – review & editing. **Hengyu Man:** Formal analysis, Visualization, Writing – original draft, Writing – review & editing. **Wenxue Cui:** Data curation, Formal analysis, Project administration, Supervision, Writing – original draft, Writing – review & editing. **Riyu Lu:** Methodology, Software, Writing – review & editing. **Chenxin Cai:** Conceptualization, Formal analysis, Methodology. **Xiaopeng Fan:** Conceptualization, Formal analysis, Methodology, Resources, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

This work was supported in part by the National Key R&D Program of China (2021YFF0900500), the National Natural Science Foundation of China (NSFC) under grants 62441202, U22B2035, 62302128, the

Postdoctoral Fellowship Program of CPSF under grant GZB20230954, and the Postdoctoral Science Foundation of Heilongjiang Province of China (LBH-Z23136). We also thank researchers in this field for establishing a theoretical foundation and providing a wealth of reference experience for our academic study.

Appendix A. Experimental details

All methods are run in each environment with 10 different random seeds, and the results are the mean success rates and the standard deviations evaluated over 15 episodes per random seed. The hyperparameters used for both continuous and discrete environments are listed below:

- Optimizer: Adam (Kingma & Ba, 2015)
- Non-linearity: ReLU
- Batch size: 512
- Target update interval: 1
- Learning rate: 10^{-4} for all network modules
- Discount factor: 0.99
- Initial temperature: 0.2

A.1. Settings for continuous environments

The hyperparameters used for the goal-conditioned robotic experiments are listed below:

- Policy/value-function network hidden layers: (256, 256, 256)
- Timesteps for data collection before training: 10,000
- Entropy target: -dim($\mathcal{A}$)
- Number of HER per transition: 5
- Target smoothing coefficient: 0.005
- Scale of the mutual information reward for MUSIC: 5000
- Dimensionality of latent option for HIDIO: 8
- Steps per option for HIDIO: 3
- HIDIO networks hidden layers: (128, 128, 128)
- Discriminator network hidden layers for HIDIO: (64, 64)
- Fixed temperature for the worker of HIDIO: 0.01
- Generator network hidden layers: (256, 256, 256)

PandaReach

- Discount factor: 0.95
- Buffer size: 500,000
- Dimensionality of the feature vector for ICM: 6
- Dimensionality of the feature vector for GeneWorker: 64
- Size of feature map for GeneWorker: 8×8
- Learning rate for GeneWorker: 3×10^{-4}
- Policy network hidden layers for GeneWorker: (64, 64)

PandaPush

- Buffer size: 3,000,000
- Dimensionality of the feature vector for ICM: 10
- Target smoothing coefficient for HIDIO: 0.001
- Dimensionality of the feature vector for GeneWorker: 64
- Size of feature map for GeneWorker: 8×8
- Target smoothing coefficient for GeneWorker: 0.05
- Learning rate for GeneWorker: 3×10^{-4}

PandaPickAndPlace

- Buffer size: 3,000,000
- Dimensionality of the feature vector for ICM: 10
- Target smoothing coefficient for HIDIO: 0.001
- Dimensionality of the feature vector for GeneWorker: 256
- Size of feature map for GeneWorker: 16×16
- Learning rate for GeneWorker: 3×10^{-4}
- Policy network hidden layers for GeneWorker: (128, 64, 64)

WebotsDoor

- Discount factor: 0.95
- Buffer size: 800,000
- Dimensionality of the feature vector for ICM: 5
- Dimensionality of the feature vector for GeneWorker: 64
- Size of feature map for GeneWorker: 8×8

A.2. Settings for discrete environments

The hyperparameters used for the classic discrete control experiments are listed below:

- Policy/value-function network hidden layers: (128, 128, 128)
- Timesteps for data collection before training: 1000
- Entropy target: $-0.98 \times \log \frac{1}{dim(\mathcal{A})}$
- Target smoothing coefficient: 0.01
- Generator network hidden layers: (128, 128, 128)
- Policy network hidden layers for GeneWorker: (64, 64, 64)
- Dimensionality of the feature vector for GeneWorker: 64
- Size of feature map for GeneWorker: 8×8
- Buffer size: 80,000

Appendix B. Network parameter sizes and training time

In this section, the trainable network parameter size and the training time of each model are listed. Each model is trained on 100 batches, with each batch containing 512 samples. The time it takes for the model to train one batch (one forward propagation and one backward propagation) is then calculated by taking the average (see Tables B.2–B.5).

Table B.2
Trainable network parameter sizes of models for continuous robotic tasks.

Method	PandaReach	PandaPush	PandaPickAndPlace	WebotsDoor
SAC (Haarnoja et al., 2019)	406,024	415,240	417,034	401,158
HIDIO (Zhang et al., 2021)	227,298	246,498	249,828	220,000
ICM (Pathak et al., 2017)	408,919	425,119	427,298	405,650
MUSIC (Zhao et al., 2021)	–	434,313	436,235	418,311
GeneWorker (ours)	430,430	502,016	549,228	487,558

Table B.3
Training time for one batch in continuous robotic tasks.

Method	PandaReach (ms)	PandaPush (ms)	PandaPickAndPlace (ms)	WebotsDoor (ms)
SAC (Haarnoja et al., 2019)	8.162	8.328	8.298	8.185
HIDIO (Zhang et al., 2021)	17.694	17.959	19.139	17.463
ICM (Pathak et al., 2017)	10.751	11.160	11.409	10.937
MUSIC (Zhao et al., 2021)	–	10.492	10.701	10.332
GeneWorker (ours)	34.517	36.225	31.901	35.631

Table B.4
Trainable network parameter sizes of models for discrete control tasks.

Method	Acrobot	CartPole	LunarLander
DQN (Mnih et al., 2015)	34,307	33,922	34,692
SAC (Christodoulou, 2019)	102,921	101,766	104,076
GeneWorker (ours)	124,835	123,714	125,956

Table B.5
Training Time for One Batch in Discrete Control Tasks.

Method	Acrobot (ms)	CartPole (ms)	LunarLander (ms)
DQN (Mnih et al., 2015)	1.855	1.865	1.865
SAC (Christodoulou, 2019)	6.902	6.981	7.032
GeneWorker (ours)	32.217	21.699	21.715

Appendix C. Visualization of neural skills

See Figs. C.9–C.20.

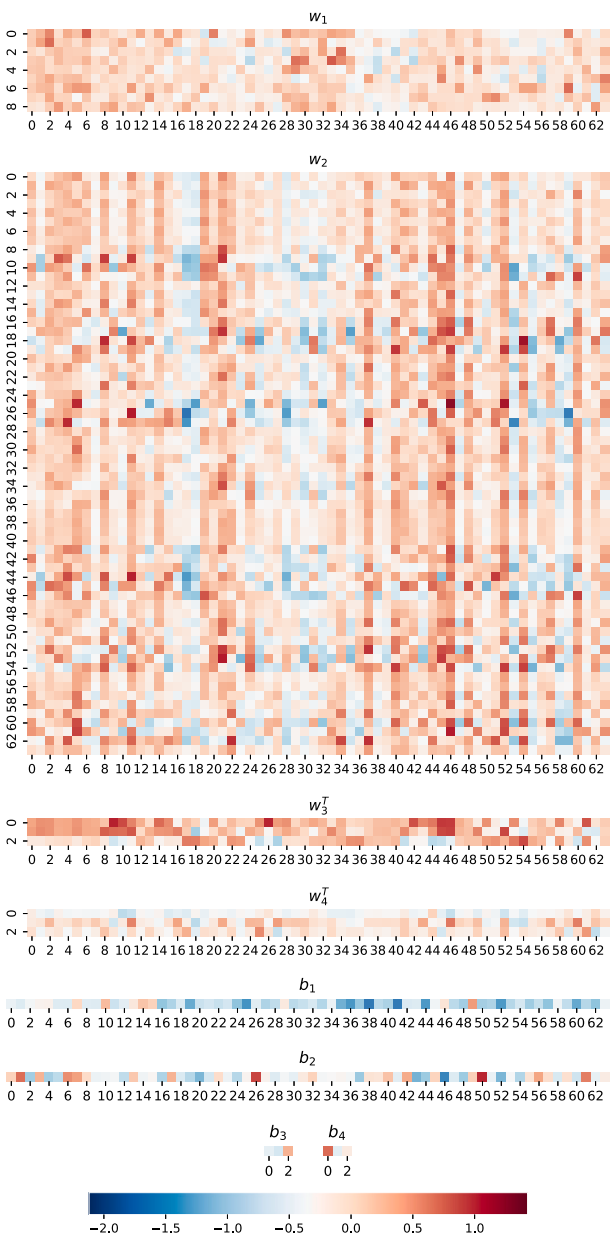


Fig. C.9. The mean values of neural skills when performing actions along the positive x-axis.

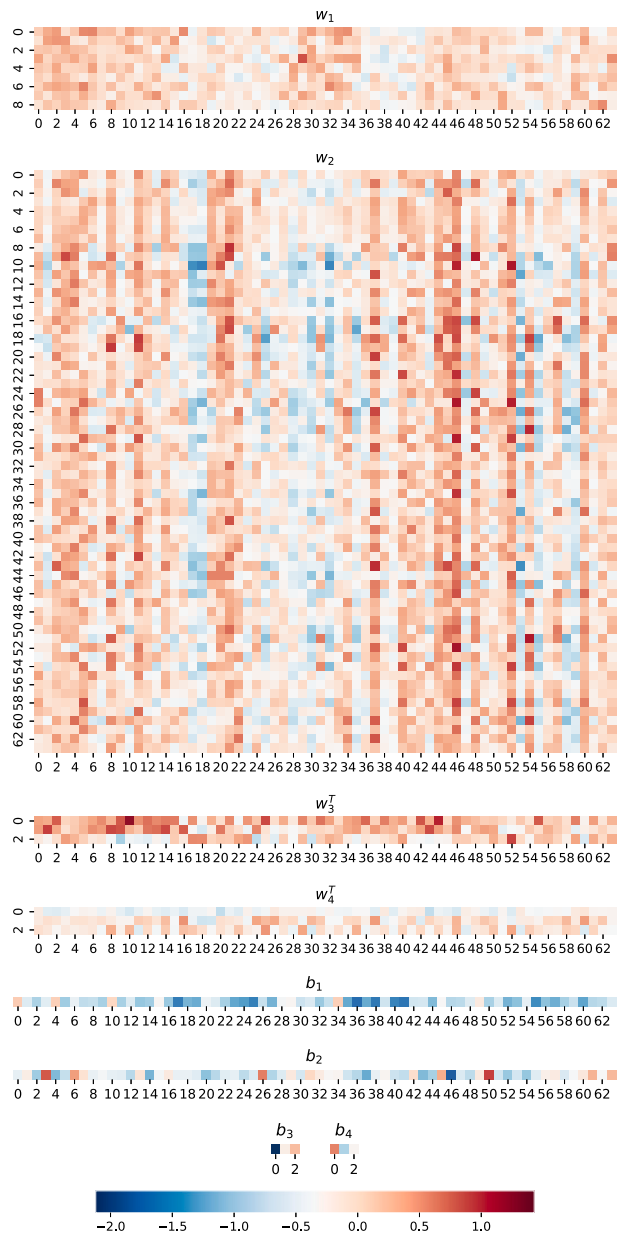


Fig. C.10. The mean values of neural skills when performing actions along the negative x-axis.

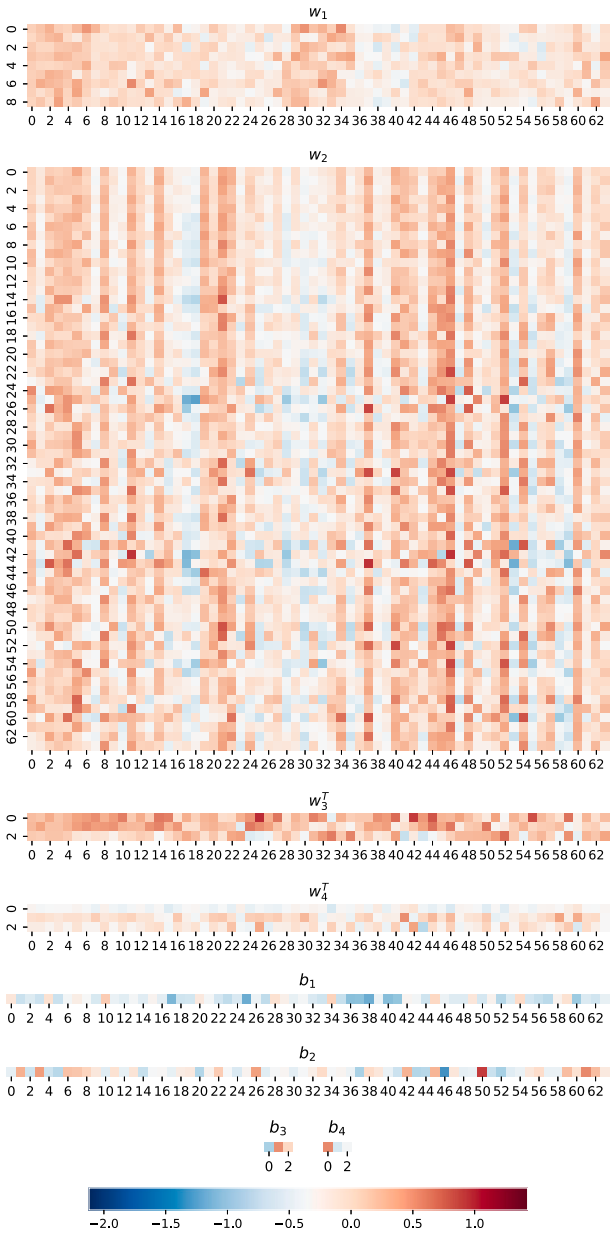


Fig. C.11. The mean values of neural skills when performing actions along the positive y-axis.

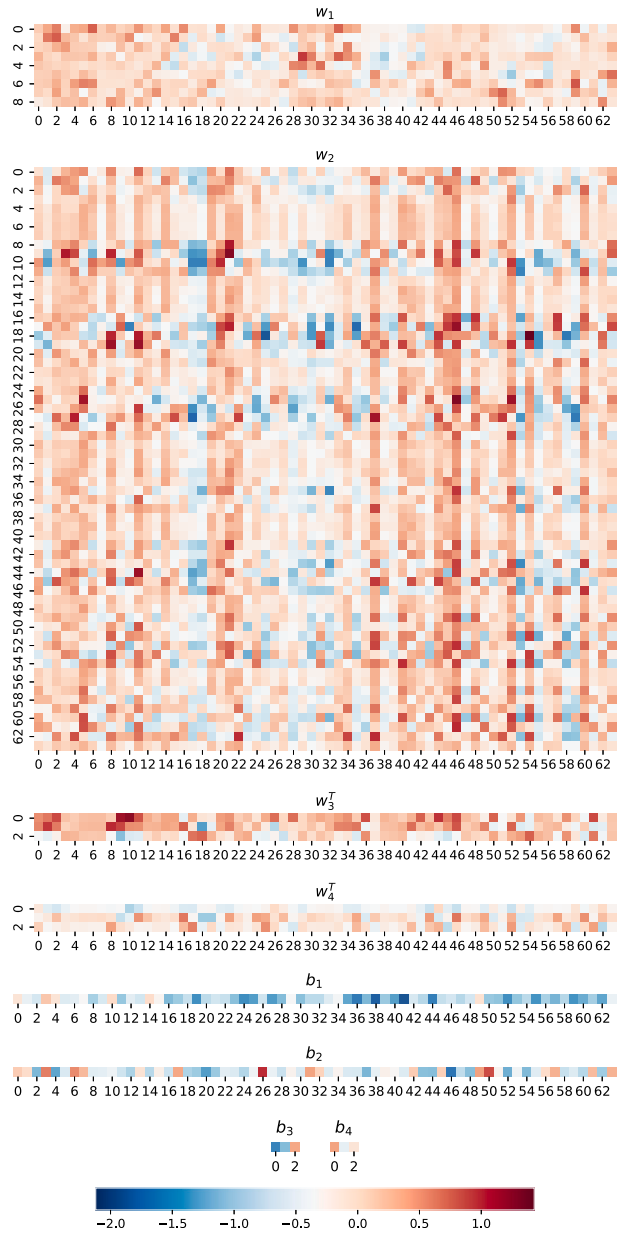


Fig. C.12. The mean values of neural skills when performing actions along the negative y-axis.

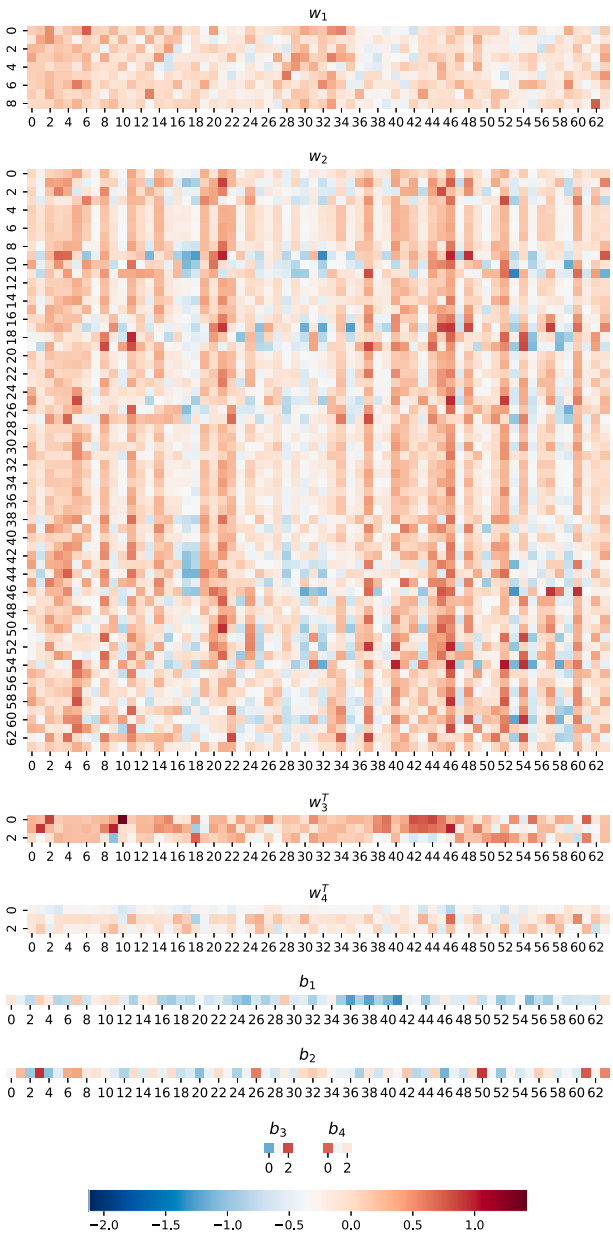


Fig. C.13. The mean values of neural skills when performing actions along the positive z-axis.

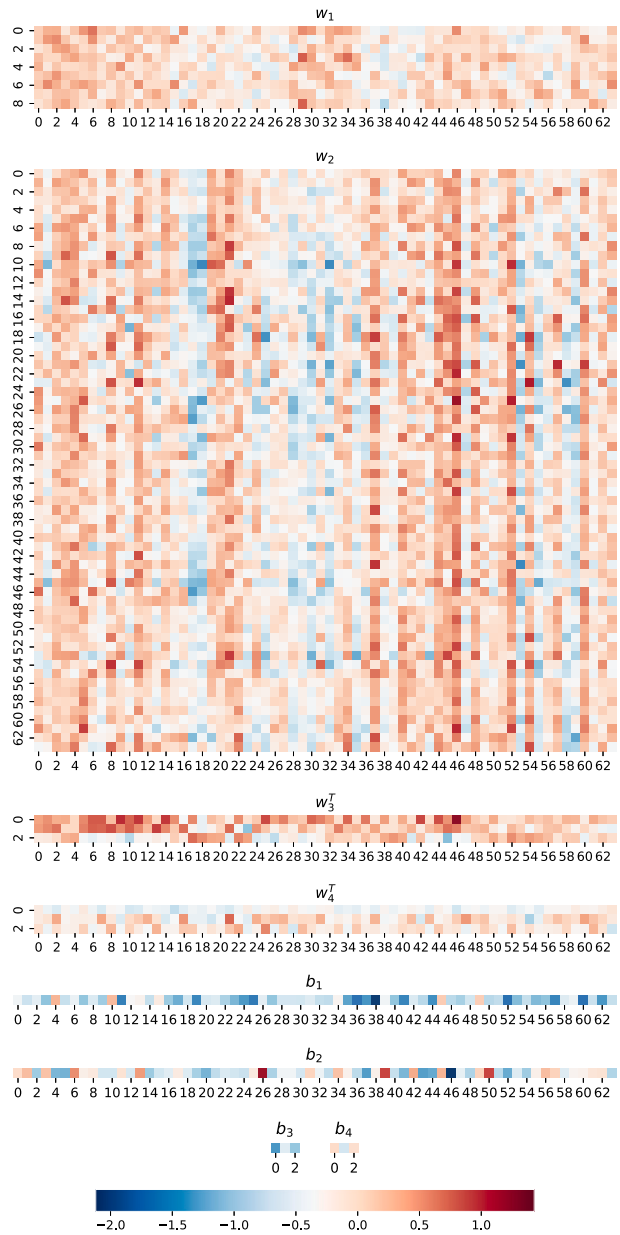


Fig. C.14. The mean values of neural skills when performing actions along the negative z-axis.

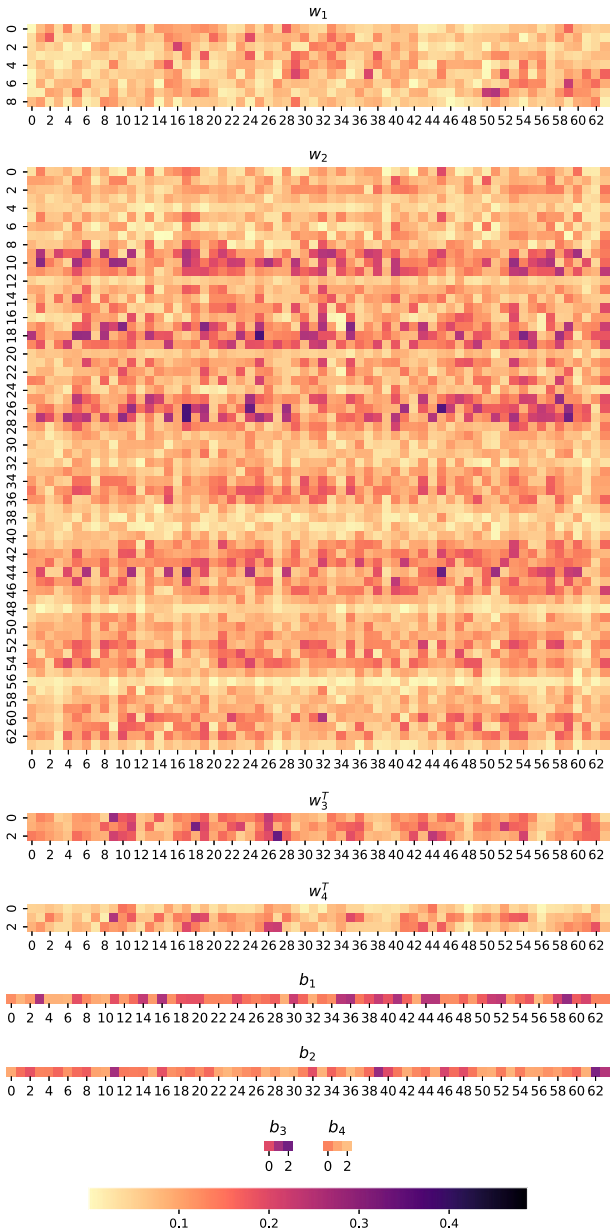


Fig. C.15. The standard deviations of neural skills when performing actions along the positive x-axis.

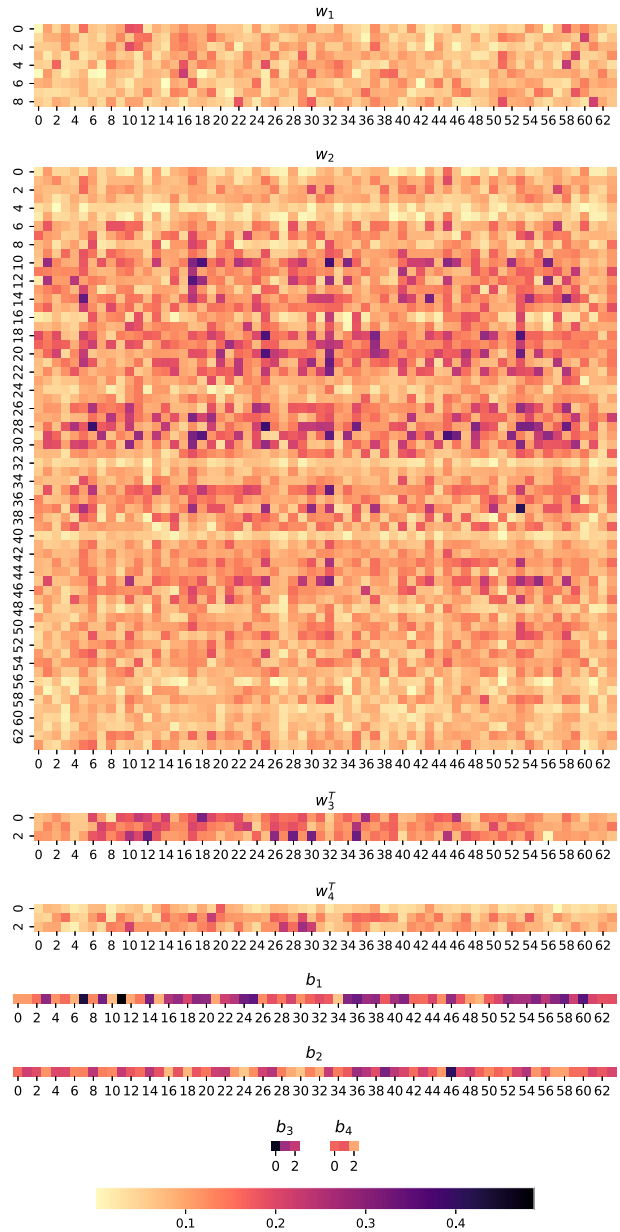


Fig. C.16. The standard deviations of neural skills when performing actions along the negative x-axis.

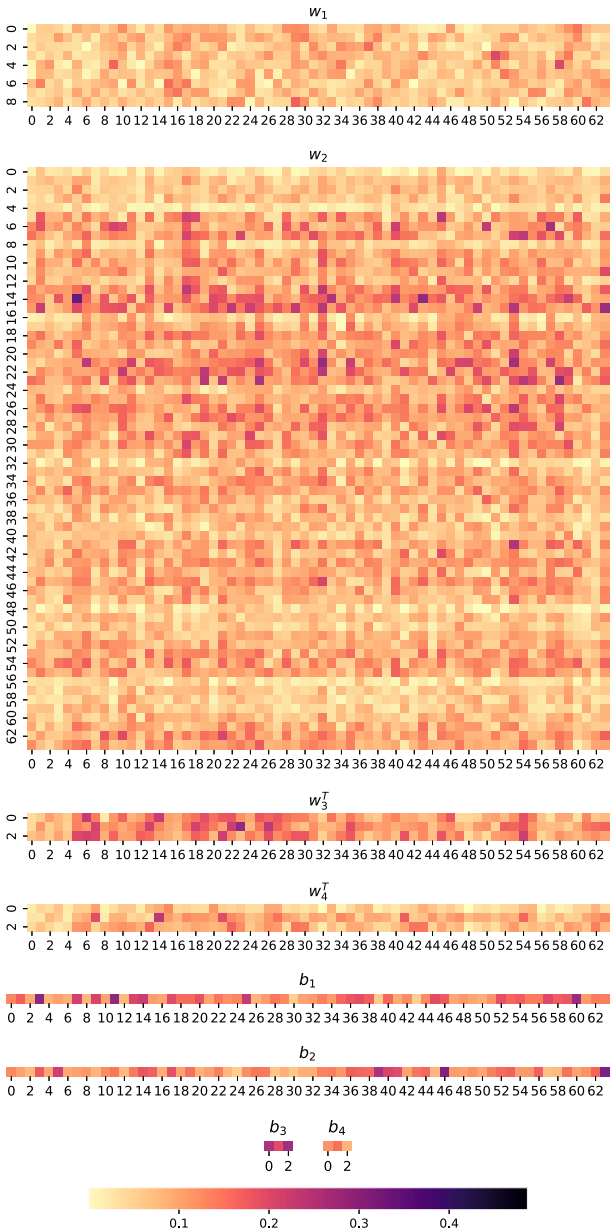


Fig. C.17. The standard deviations of neural skills when performing actions along the positive y-axis.

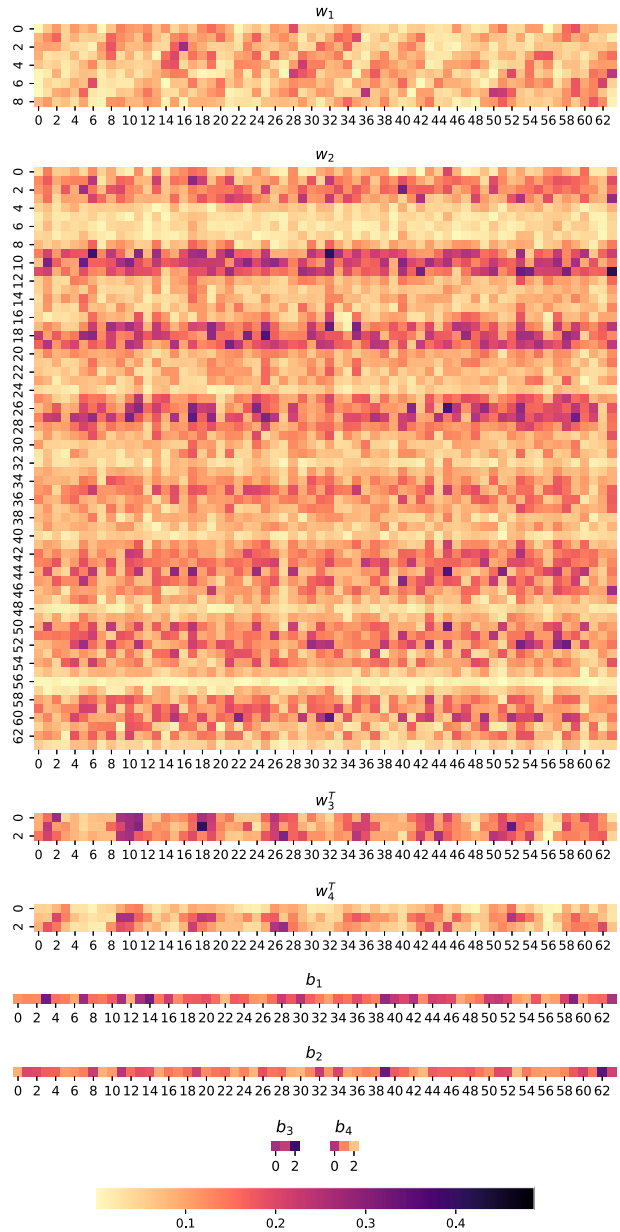


Fig. C.18. The standard deviations of neural skills when performing actions along the negative y-axis.

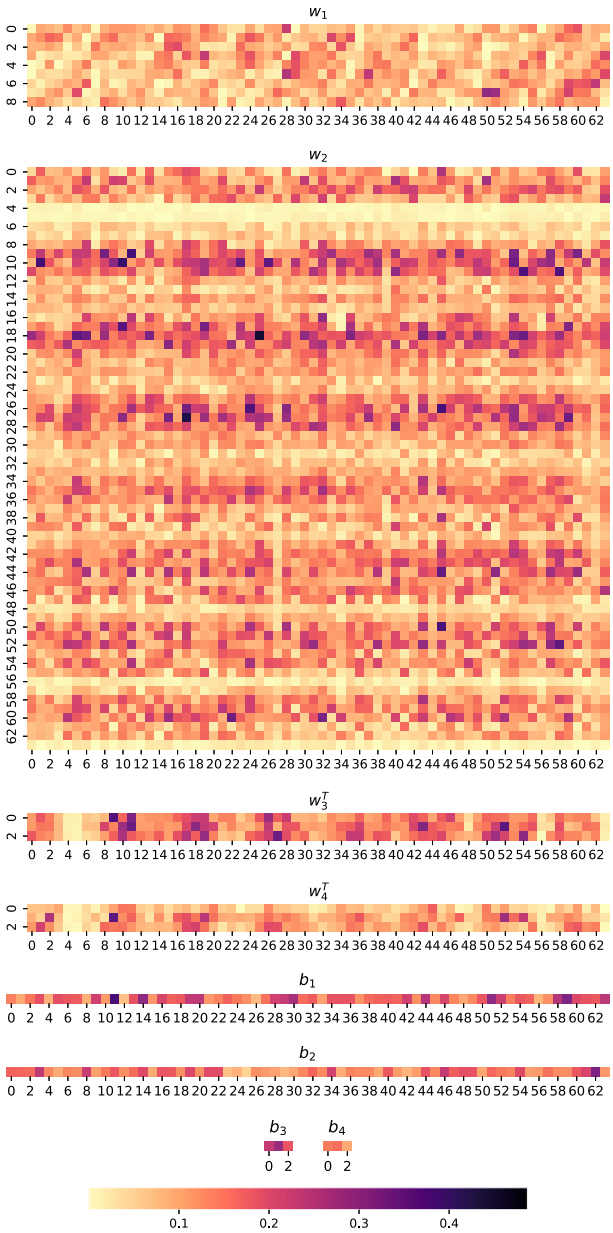


Fig. C.19. The standard deviations of neural skills when performing actions along the positive z-axis.

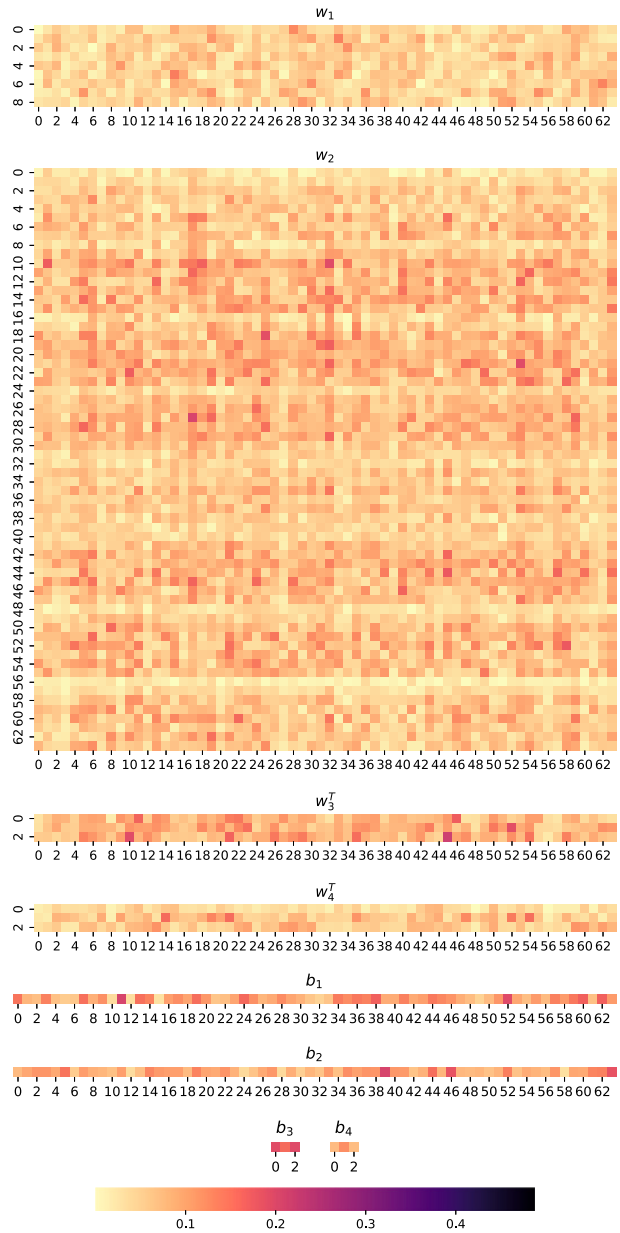


Fig. C.20. The standard deviations of neural skills when performing actions along the negative z-axis.

References

Agrawal, P., Nair, A. V., Abbeel, P., Malik, J., & Levine, S. (2016). Learning to poke by poking: Experiential learning of intuitive physics. *Advances in Neural Information Processing Systems*, *29*.

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., et al. (2017). Hindsight experience replay. *Advances in Neural Information Processing Systems*, *30*.

Bagaria, A., Senthil, J. K., & Konidaris, G. (2021). Skill discovery for exploration and planning using deep skill graphs. In *International conference on machine learning* (pp. 521–531). PMLR.

Barense, M. D., Warren, J. D., Bussey, T. J., & Saksida, L. M. (2016). The temporal lobes. In *Oxford textbook of cognitive neurology and dementia* (pp. 39–50). Oxford University Press.

Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man and Cybernetics*, (5), 834–846.

Brodal, P. (2016). The Motor Cortical Areas and descending pathways. In *The central nervous system*. Oxford University Press.

Campos, V., Trott, A., Xiong, C., Socher, R., Giró-i Nieto, X., & Torres, J. (2020). Explore, discover and learn: Unsupervised discovery of state-covering skills. In *International conference on machine learning* (pp. 1317–1327). PMLR.

Chane-Sane, E., Schmid, C., & Laptev, I. (2021). Goal-conditioned reinforcement learning with imagined subgoals. In *International conference on machine learning* (pp. 1430–1440). PMLR.

Christodoulou, P. (2019). Soft actor-critic for discrete action settings. arXiv preprint arXiv:1910.07207.

Eysenbach, B., Gupta, A., Ibarz, J., & Levine, S. (2019). Diversity is all you need: Learning skills without a reward function. In *International conference on learning representations*.

Finn, C., Levine, S., & Abbeel, P. (2016). Guided cost learning: Deep inverse optimal control via policy optimization. In *International conference on machine learning* (pp. 49–58). PMLR.

Fu, J., Luo, K., & Levine, S. (2018). Learning robust rewards with adverserial inverse reinforcement learning. In *International conference on learning representations*.

Fujimoto, S., Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In *International conference on machine learning* (pp. 1587–1596). PMLR.

Gallouédec, Q., Cazin, N., Dellandréa, E., & Chen, L. (2021). panda-gym: Open-source goal-conditioned environments for robotic learning. In *4th robot learning workshop: self-supervised and lifelong learning*.

Gregor, K., Rezende, D. J., & Wierstra, D. (2016). Variational intrinsic control. arXiv preprint arXiv:1611.07507.

Gupta, A. (2017). Occipital lobe. In J. Vonk, & T. Shackelford (Eds.), *Encyclopedia of animal cognition and behavior* (pp. 1–2). Cham: Springer International Publishing.

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning* (pp. 1861–1870). PMLR.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., et al. (2019). Soft actor-critic algorithms and applications. arXiv preprint arXiv:1812.05905v2.

Ho, J., & Ermon, S. (2016). Generative adversarial imitation learning. *Advances in Neural Information Processing Systems*, *29*.

Jain, D., Iscen, A., & Caluwaerts, K. (2019). Hierarchical reinforcement learning for quadruped locomotion. In *2019 IEEE/RSJ international conference on intelligent robots and systems* (pp. 7551–7557). IEEE.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International conference on learning representations*.

Kingma, D. P., & Welling, M. (2022). Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114v11.

Klocek, S., Maziarka, Ł., Wołczyk, M., Tabor, J., Nowak, J., & Śmieja, M. (2019). Hypernetwork functional image representation. In *Artificial neural networks and machine learning–ICANN 2019: workshop and special sessions: 28th international conference on artificial neural networks, Munich, Germany, September 17–19, 2019, proceedings 28* (pp. 496–510). Springer.

Levine, S., Pastor, P., Krizhevsky, A., Ibarz, J., & Quillen, D. (2018). Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*, *37*(4–5), 421–436.

Levy, A., Konidaris, G., Platt, R., & Saenko, K. (2019). Learning multi-level hierarchies with hindsight. In *Proceedings of international conference on learning representations*.

Li, A., Florensa, C., Clavera, I., & Abbeel, P. (2020). Sub-policy adaptation for hierarchical reinforcement learning. In *International conference on learning representations*.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., et al. (2015). Continuous control with deep reinforcement learning.

Michel, O. (2004). Webots: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, *1*(1), 39–42.

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., et al. (2016). Asynchronous methods for deep reinforcement learning. In *International conference on machine learning* (pp. 1928–1937). PMLR.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*(7540), 529–533.

Nachum, O., Gu, S. S., Lee, H., & Levine, S. (2018). Data-efficient hierarchical reinforcement learning. *Advances in Neural Information Processing Systems*, *31*.

Nair, A. V., Pong, V., Dalal, M., Bahl, S., Lin, S., & Levine, S. (2018). Visual reinforcement learning with imagined goals. *Advances in Neural Information Processing Systems*, *31*.

Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning* (pp. 2778–2787). PMLR.

Pathak, D., Mahmoudieh, P., Luo, M., Agrawal, P., Chen, D., Shentu, F., et al. (2018). Zero-shot visual imitation. In *International conference on learning representations*.

Pong, V. H., Dalal, M., Lin, S., Nair, A., Bahl, S., & Levine, S. (2020). Skew-fit: state-covering self-supervised reinforcement learning. In *Proceedings of the 37th international conference on machine learning* (pp. 7783–7792).

Precup, R.-E., Roman, R.-C., & Safaei, A. (2021). *Data-driven model-free controllers*. CRC Press.

Qian, Z., You, M., Zhou, H., Xu, X., & He, B. (2023). Goal-conditioned reinforcement learning with disentanglement-based reachability planning. *IEEE Robotics and Automation Letters*.

Rigatos, G., Siano, P., Selisteanu, D., & Precup, R. (2017). Nonlinear optimal control of oxygen and carbon dioxide levels in blood. *Intelligent Industrial Systems*, *3*, 61–75.

Sharma, A., Gu, S., Levine, S., Kumar, V., & Hausman, K. (2020). Dynamics-aware unsupervised discovery of skills. In *International conference on learning representations*.

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. In *International conference on machine learning* (pp. 387–395). Pmlr.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., et al. (2017). Mastering the game of go without human knowledge. *Nature*, *550*(7676), 354–359.

Silver, D., Singh, S., Precup, D., & Sutton, R. S. (2021). Reward is enough. *Artificial Intelligence*, *299*, Article 103535.

Singh, A., Yang, L., Hartikainen, K., Finn, C., & Levine, S. (2019). End-to-end robotic reinforcement learning without reward engineering. arXiv preprint arXiv:1904.07854.

Su, S., Yan, Q., Zhu, Y., Zhang, C., Ge, X., Sun, J., et al. (2020). Blindly assess image quality in the wild guided by a self-adaptive hyper network. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 3667–3676).

Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, *12*.

Tessler, C., Givony, S., Zahavy, T., Mankowitz, D., & Mannor, S. (2017). A deep hierarchical approach to lifelong learning in minecraft. In *Proceedings of the AAAI conference on artificial intelligence, vol. 31, no. 1*.

Towers, M., Terry, J. K., Kwiatkowski, A., Balis, J. U., Cola, G. d., Deleu, T., et al. (2023). *Gymnasium*. Zenodo.

Ucgun, H., Okten, I., Yuzgec, U., & Kesler, M. (2022). Test platform and graphical user interface design for vertical take-off and landing drones. *Romanian Journal of Information Science and Technology*, *25*(3), 350–367.

Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning* (pp. 1995–2003). PMLR.

Wu, Y., Xie, X.-J., & Hou, Z.-G. (2022). Adaptive fuzzy asymptotic tracking control of state-constrained high-order nonlinear time-delay systems and its applications. *IEEE Transactions on Cybernetics*, *52*(3), 1671–1680.

Wulfmeier, M., Ondruska, P., & Posner, I. (2015). Maximum entropy deep inverse reinforcement learning. arXiv preprint arXiv:1507.04888.

Zhang, J., Yu, H., & Xu, W. (2021). Hierarchical reinforcement learning by discovering intrinsic options. In *International conference on learning representations*.

Zhao, R., Gao, Y., Abbeel, P., Tresp, V., & Xu, W. (2021). Mutual information state intrinsic control. In *International conference on learning representations*.